IBM RAG & AGENTIC AI PROFESSIONAL CERTIFICATE · COURSE 8

Introduction to Containers w/ Docker, Kubernetes & OpenShift

Course 8 of the IBM RAG and Agentic AI Professional Certificate — sample build

Compiled for Shivam · Edition of 01 September 2026

Contents

Module 1 — Containers and Why They Exist

1.1 The problem before containers

Before containers, deploying an application meant hoping the production server had the exact same OS libraries, runtime version, and configuration as your laptop. "It works on my machine" was not a joke — it was the default state of software deployment.

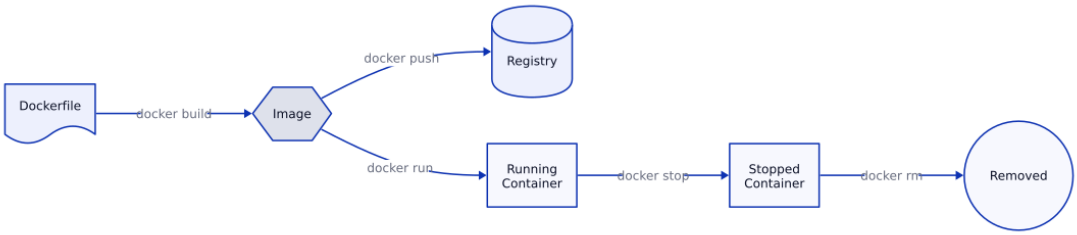
> **THE CORE INSIGHT**
>
> A container packages an application **together with everything it needs to run** — libraries, runtime, system tools, settings — into a single, portable unit. Unlike a virtual machine, it does not include a full guest operating system; it shares the host kernel and isolates only the process, filesystem, and network namespace.

1.2 Containers vs virtual machines

Aspect	Container	Virtual Machine
Boot time	Milliseconds to seconds	Tens of seconds to minutes
OS overhead	Shares host kernel	Full guest OS per instance
Image size	Typically MBs	Typically GBs
Isolation strength	Process-level	Hardware-level

> **LIKELY QUESTION**
>
> **Q: Why are containers faster to start than VMs?** A: Containers don't boot a kernel — they start a process using the host kernel's namespaces and cgroups for isolation. A VM has to boot an entire guest operating system before your application even starts.

1.3 The container lifecycle



> **WHEN TO USE WHAT**
>
> - Use a **Dockerfile** when you need a reproducible build from source.
> - Use `docker run` directly against a public image when you just need a dependency (e.g. a Postgres instance for local dev) and don't need to customize it.
> - Push to a registry only once the image is something other services or environments need to pull — not for pure local iteration.

1.4 A minimal Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "main.py"]
```

Each instruction creates a new, cached layer. Reordering `COPY . .` to come *before* `pip install` is a common mistake — it invalidates the dependency-install cache on every code change, making rebuilds far slower than necessary.

Module 2 — Kubernetes Fundamentals

2.1 Why Kubernetes exists

Docker solves running one container reliably. Kubernetes solves running **hundreds of containers across many machines**, reliably, with automatic recovery when things fail.

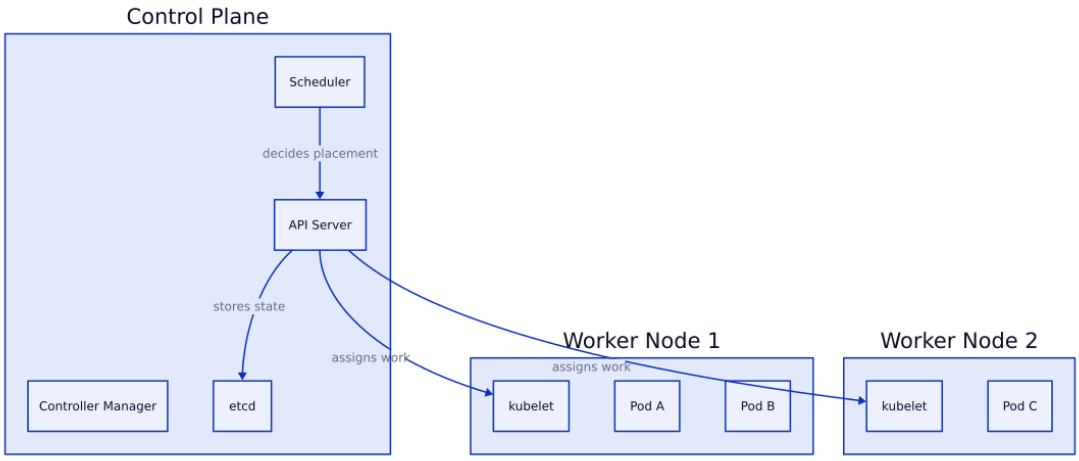
> **THE ORCHESTRATION PROBLEM**
>
> Once you have more than a handful of containers, you need answers to questions Docker alone doesn't address: Which machine should this container run on? What happens when that machine dies? How do containers find each other over the network? How do I roll out a new version without downtime? Kubernetes is, fundamentally, a control loop that continuously reconciles "what you asked for" against "what's actually running."

2.2 Core objects

Object	Purpose
Pod	Smallest deployable unit — one or more containers sharing network/storage
Deployment	Manages a set of identical Pod replicas, handles rolling updates
Service	Stable network identity for a set of Pods, even as they're replaced
ConfigMap / Secret	External configuration and sensitive values, injected into Pods

> **LIKELY QUESTION**
>
> **Q: What's the difference between a Pod and a Deployment?** A: A Pod is a single running instance of your container(s). It's disposable — Kubernetes may kill and recreate it at any time. A Deployment is a higher-level object that *manages* Pods: it specifies how many replicas should exist and handles creating, replacing, and rolling out new versions of them. You almost never create a bare Pod directly in production; you create a Deployment and let it manage Pods.

2.3 The architecture



WHEN TO USE WHAT

- Use a **Deployment** for stateless services (most web apps, APIs).
- Use a **StatefulSet** when Pods need stable identities and storage (databases, message queues).
- Use a **DaemonSet** when you need exactly one Pod per node (log collectors, monitoring agents).

2.4 A minimal Deployment manifest

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-api
  template:
    metadata:
      labels:
        app: my-api
    spec:
      containers:
        - name: my-api
          image: my-registry/my-api:1.2.0
          ports:
            - containerPort: 8080
```

`replicas: 3` tells Kubernetes to keep exactly three identical Pods running at all times. If one crashes, the Controller Manager notices the mismatch between desired (3) and actual (2) state, and schedules a replacement — no manual intervention required.